

Chat Context Export

Platform: ChatGPT
Exported: 13/09/2026, 15:24:05
Conversation ID: 6a9eeac8-99b4-83ee-9368-4e7cbc22896b

Conversation

User

2. Theory: How TLS Fingerprinting Works This section is deliberately mechanical. Every claim below maps to a specific byte offset our parsers rely on. ### 2.1 The TLS Record Layer (outermost framing) Every TLS message — Handshake, Alert, ChangeCipherSpec, Application Data — is wrapped in a 5-byte **Record** header: We only care about records with Content Type . Everything else (most commonly ChangeCipherSpec, which persists as a TLS 1.3 "middlebox compatibility" artifact even though it's cryptographically meaningless in 1.3) must be **skipped**, not **dropped** — the stream continues, and a Handshake record can immediately follow. ### 2.2 The Handshake Layer Inside a Content-Type-22 record sits one or more **Handshake** messages, each with its own 4-byte header: The 24-bit length exists specifically because certificate chains can exceed a single TLS record's ~16 KB fragment limit — so a **handshake message** boundary and a **TLS record** boundary are not the same thing. A handshake message can legitimately span multiple records, and a single record can legitimately contain multiple handshake messages back-to-back. Any correct implementation must handle both. ### 2.3 ClientHello Body — the JA3 Source Fields Extensions are themselves nested TLV structures. The ones we parse: | Ext Type | Name | Contribution | | ----- | ----- | ----- | | | SNI | Hostname (for display / JA4's/ indicator) | | | Supported Groups | JA3 Field 4 (elliptic curves) | | | EC Point Formats | JA3 Field 5 | | | Signature Algorithms | JA4_c wire-order tail | | | ALPN | JA4_a first+last char | | | Supported Versions | JA4's real version source | **Key subtlety exploited by our parser:** in TLS 1.3, is deliberately frozen at (RFC 8446 §4.1.2, middlebox compatibility). The **actual** max version a client supports lives only inside the extension. JA3 does not care about this — it hashes the frozen body version anyway (a documented JA3 limitation). JA4 explicitly prefers and falls back to the body version only if the extension is absent. ### 2.4 ServerHello Body — the JA3S Source Fields Structurally identical up through the cipher field, except the server selects **one** cipher, not a list: Asymmetry worth noting in the report: the extension () is a **list** in ClientHello but exactly **2 bytes** (the single negotiated version) in ServerHello. Our parsers handle each side with dedicated logic rather than one shared extension-parsing routine, because the two are not the same wire format despite sharing an extension type code. ### 2.5 GREASE (RFC 8701) Clients (Chrome-family browsers especially) insert meaningless placeholder values shaped like (, , ...) into cipher suites, extensions, groups, and ALPN lists. This is intentional protocol hygiene — it stops middleboxes from ossifying around a fixed set of "known" values. For fingerprinting, GREASE is pure noise and **must** be filtered before hashing, or the same physical client would produce a different hash on every connection. ### 2.6 The Fingerprint Computation **JA3**: 0 MD5, wire order

preserved throughout. **JA3S**: MD5. **JA4**: A structured 3-part string — a human-readable 10-char prefix (protocol/version/SNI-flag/counts/ALPN) plus two truncated-SHA256 hashes, one over *sorted* ciphers and one over *sorted* extensions concatenated with *wire-order* signature algorithms. The sort is the deliberate fix for browser randomization (§6 below). **JA4S**: Same idea, 7-char prefix, raw 4-hex-digit selected cipher (not hashed — there's only one value, nothing to sort), and a sorted-extension hash.

— Tell me is the theory correct

Code Block 1 (text)

Offset	Size	Field
0	1	Content Type (20=ChangeCipherSpec, 21=Alert, 22=Handshake, 23=ApplicationData)
1–2	2	Legacy Version (e.g. 0x0301, 0x0303 — frozen in TLS 1.3, must be ignored per JA3 spec)
3–4	2	Fragment Length
5..	var	Fragment payload

Code Block 2 (text)

Offset	Size	Field
0	1	Message Type (1 = ClientHello, 2 = ServerHello, 11 = Certificate, ...)
1–3	3	Length (24-bit big-endian — up to 16 MB, to accommodate large cert chains)
4..	var	Handshake body

Code Block 3 (text)

Offset	Field	JA3 Field?
0–1	Client Version (2B)	Field 1 (legacy body version, NOT record version)
2–33	Random (32B)	— (skipped)
34	Session ID Length (1B)	—
35..(35+N-1)	Session ID (NB)	— (skipped)
(35+N)	Cipher Suites Len (2B)	—
(37+N)..	Cipher Suites (MB)	Field 2 (array of 2B cipher IDs)
...	Compression Methods	— (skipped, always null post-TLS1.3)
...	Extensions Length (2B)	—
...	Extensions (TLV chain)	Field 3 = extension <i>type codes</i> in wire order

Code Block 4 (text)

Offset	Field	JA3S Field?
0–1	Server Version (2B)	Field 1
2–33	Random (32B)	— (skipped)
34	Session ID Length (1B)	—
(35+N)	Selected Cipher (2B)	Field 2 (single value, not an array)
(37+N)	Compression Method (1B)	— (skipped)
...	Extensions	Field 3

Assistant

Yes — the theory is largely correct, and the structure is good for a technical report. But I would fix a few

points before you treat it as authoritative, especially around TLS record fragmentation, TLS 1.3 compression methods, GREASE, and the exact JA4 construction.

1. TLS Record Layer — correct, with one wording change Your 5-byte record header is correct: 1 byte ContentType 2 bytes legacy_record_version 2 bytes length payload And you're right that your parser should skip non-handshake records and continue processing the TCP stream. One important distinction: "Everything else ... must be skipped, not dropped" I'd phrase this as "non-Handshake records are ignored for ClientHello/ServerHello extraction, but their bytes must still be consumed so stream parsing continues." That's more precise. For example, a ChangeCipherSpec record shouldn't cause you to discard the TCP flow.
2. Handshake Layer — correct and important This section is correct. The 4-byte handshake header is: And yes, TLS record boundaries and handshake-message boundaries are independent. This is particularly important for your C++ capture engine because you cannot assume: You can encounter: or: So your TCP reassembly → TLS record parsing → handshake-message parsing architecture is the right conceptual model. One tiny correction: "to accommodate large cert chains" The 24-bit handshake length is the generic Handshake protocol length, not something introduced specifically for certificate chains. Certificate messages are one reason large handshake messages matter. I'd change that sentence.
3. ClientHello — mostly correct Your offsets are correct for the ClientHello structure. The important distinction you've made is also correct: JA3 uses the ClientHello , not the TLS record version. And: TLS 1.3 clients normally put in ClientHello's legacy_version and communicate the actual supported versions through . That's an important point for your report. Your extension table is also broadly correct: ExtensionJA3JA4/otherSNI Extension IDSNI indicator / hostnameSupported Groups Curvesrelevant JA4 dataEC Point Formats Point formats—Signature Algorithms —JA4 signature-algorithm componentALPN Extension ID onlyJA4 ALPN componentSupported Versions Extension ID onlyactual TLS version One thing I'd change: "Compression Methods — skipped, always null post-TLS1.3" That's too strong. TLS 1.3 does not negotiate compression, and the ClientHello compression_methods field is still present because the ClientHello structure retains the TLS 1.2-era field. But don't call it "always null"; the field is a vector containing a compression method value, and TLS 1.3 requires () for compatibility. So write: "Compression Methods — not used for JA3; TLS 1.3 requires the null compression method."
4. ServerHello — correct Your ServerHello description is good. The important asymmetry is exactly what you identified: For example: So having separate ClientHello and ServerHello parsing logic is sensible.
5. GREASE — correct, but there's one thing I'd tighten Your description is basically right. The GREASE values are of the form: and GREASE can appear in multiple extensible protocol fields. You're also right about the fingerprinting consequence: GREASE should be removed before computing JA3/JA4 components where the specification requires GREASE filtering. I'd avoid saying simply: "GREASE ... must be filtered before hashing" because exactly what gets filtered depends on the fingerprint algorithm/component. Your implementation should explicitly define GREASE filtering at the appropriate parsing points.
6. JA3 — correct This is correct: followed by: And importantly, JA3 preserves the order observed on the wire. For example: Then MD5 of that exact comma-separated representation. That's a very important difference from JA4.
7. JA3S — correct Also correct: → MD5. And the ServerHello has only one selected cipher, rather than the ClientHello cipher list.
8. JA4 — this is the part I would revise most Your conceptual description is good: human-readable prefix + two truncated SHA-256 components

and you're correctly highlighting the difference from JA3: JA3: wire order matters JA4: certain components are sorted However, this sentence is potentially misleading: "one over sorted ciphers and one over sorted extensions concatenated with wire-order signature algorithms." The exact JA4 construction should be stated according to the particular JA4 specification/version you're implementing, because the components and their ordering are quite specific. I'd avoid describing it as simply: without showing the actual construction. For a technical report, I'd recommend writing the actual formula used by your implementation, e.g.: provided that matches the JA4 specification/version you're implementing. This is one place where I would cross-check your implementation against the canonical JA4 specification before publishing the paragraph.

9. JA4S — also needs slightly more precision Your high-level statement is good: 7-character prefix + raw selected cipher + sorted extension hash. But again, I'd document the exact prefix fields rather than saying only "same idea." JA4S isn't simply: There are specific fields in its prefix and hash construction. One more important issue: TCP fragmentation Because this is describing your fingerprinting engine, I would add a subsection between Record Layer and Handshake Layer: TCP Stream Reassembly TLS runs over TCP, so the bytes arriving from are not guaranteed to correspond to TLS records. For example: or: Therefore the parser should conceptually operate as: This is very important for your engine because otherwise a perfectly valid fragmented ClientHello can be classified as malformed or unknown. Overall verdict I'd rate your current theory ~90–95% correct conceptually. The architecture and major protocol facts are right. Before putting it into a formal report, I'd make these changes: Clarify that non-handshake records are consumed/ignored, not discarded from the stream. Change the 24-bit length explanation — it isn't specifically "because of certificate chains." Change to a more precise TLS 1.3 statement. Qualify GREASE filtering according to the fingerprint specification/component. Expand the exact JA4/JA4S construction, because that's the area where an implementation/report can easily become subtly incorrect. Add TCP stream reassembly, because TLS record boundaries do not correspond to packet boundaries. With those changes, this would be a strong theory section for your TLS fingerprinting engine documentation, particularly because you're explicitly tying protocol structure to the offsets your C++ parser consumes.

Code Block 1 (text)

```
1 byte HandshakeType  
3 bytes uint24 length
```

Code Block 2 (text)

```
[Record][complete ClientHello]
```

Code Block 3 (text)

```
Record 1:  
Handshake header + first 5000 bytes  
  
Record 2:  
remaining ClientHello bytes
```

Code Block 4 (text)

```
Record:  
ClientHello  
ServerHello  
...
```

Code Block 5 (text)

```
ClientHello:  
supported_versions = vector of versions  
  
ServerHello:  
supported_versions = single selected version
```

Code Block 6 (text)

```
ClientHello:  
43 02 03 04 03 03 ...  
☒  
length = 2  
03 04  
03 03  
  
ServerHello:  
2b 00 02 03 04  
☒  
selected TLS 1.3
```

Code Block 7 (text)

```
0x0A0A  
0x1A1A  
0x2A2A  
...  
0xFAFA
```

Code Block 8 (text)

```
TLSVersion,  
Ciphers,  
Extensions,  
Curves,  
PointFormats
```

Code Block 9 (text)

```
MD5
```

Code Block 10 (text)

```
771,  
4865-4866-4867,  
0-11-10-35-16-5-13,  
29-23-24,  
0
```

Code Block 11 (text)

```
TLSVersion,  
SelectedCipher,  
Extensions
```

Code Block 12 (text)

```
sorted extensions + wire-order signature algorithms
```

Code Block 13 (text)

```
JA4 =  
prefix +  
SHA256(sorted cipher component)[:12] +  
SHA256(sorted extension component + signature algorithms)[:12]
```

Code Block 14 (text)

```
JA4 but for ServerHello
```

Code Block 15 (text)

```
TCP packet 1:  
[first 200 bytes of TLS record]  
  
TCP packet 2:  
[remaining TLS record]
```

Code Block 16 (text)

```
TCP packet:  
[multiple TLS records]
```

Code Block 17 (text)

```
TCP packets  
☒  
TCP stream reassembly  
☒  
TLS Record parser  
☒  
Handshake message parser
```

```

ClientHello / ServerHello parser
GREASE filtering
JA3 / JA3S / JA4 / JA4S
```

User

A handshake message can legitimately span multiple records, and a single record can legitimately contain multiple handshake messages back-to-back. does it mean that from a single packet multiple tbs connections can be made

Assistant

Not quite. The key distinction is between a network packet, a TLS record, and a TCP connection. Think of the hierarchy A single TCP connection can contain many TLS handshake messages. For example: What your sentence means "A handshake message can legitimately span multiple records" means one TLS handshake message can be split across multiple TLS records: This is still one ClientHello, not two connections. Conversely: "a single record can legitimately contain multiple handshake messages" means: Again, this is one TCP connection, containing multiple TLS handshake messages. So can one packet contain multiple connections? Normally, no. A captured Ethernet/IP/TCP packet belongs to one TCP flow, identified by its 5-tuple: For example: That packet belongs to that one TCP connection/flow. What can happen is: So your parser could encounter something like: That's one packet → one TCP connection → multiple TLS records → multiple handshake messages. And the reverse is also possible: So for your TLS fingerprinting engine, don't create a fingerprint per packet. You should reconstruct the TCP byte stream and then parse TLS from that stream.

Code Block 1 (text)

```

TCP connection
→
many TCP packets
→
TCP byte stream
→
many TLS records
→
many TLS handshake messages
```

Code Block 2 (text)

```

TCP connection #1
→
→ → TCP packet #1
```

```

[ ] [ ] TLS Record
[ ] [ ] Handshake: ClientHello
[ ] [ ] Handshake: ???
[ ]
[ ] [ ] TCP packet #2
[ ] [ ] [ ] TLS Record
[ ] [ ] [ ] Handshake: ServerHello
[ ]
[ ] [ ] TCP packet #3
[ ] [ ] TLS Record
[ ] [ ] Handshake: Certificate

```

Code Block 3 (text)

TLS Record #1
[] first part of ClientHello
[]
TLS Record #2
[] remaining part of ClientHello

Code Block 4 (text)

TLS Record #1
ClientHello
ServerHello
Certificate

Code Block 5 (text)

(source IP, source port,
destination IP, destination port,
protocol)

Code Block 6 (text)

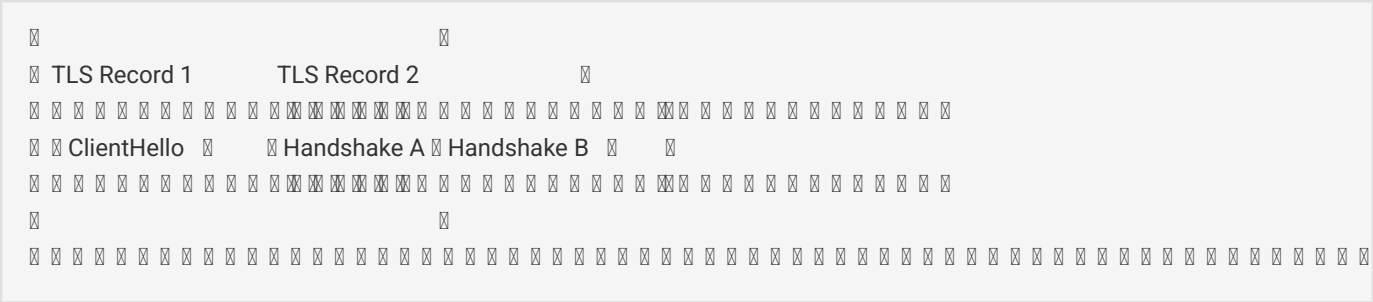
192.168.1.10:52341 → 142.250.72.14:443 TCP

Code Block 7 (text)

- ONE TCP packet
 - contains TCP payload
 - contains multiple TLS records
 - each TLS record may contain multiple handshake messages

Code Block 8 (text)

TCP Packet



Code Block 9 (text)

```
ClientHello
TLS Record 1
TCP packet 1

TLS Record 2
TCP packet 2

TLS Record 3
TCP packet 3
```

User

sudo apt update sudo apt install -y build-essential cmake libpcap-dev libssl-dev redis-server python3-venv
sudo systemctl enable --now redis-server redis-cli ping give me equivalent mac

Assistant

On macOS, the closest equivalent uses Homebrew instead of , and instead of . Assuming you already have Homebrew: You should get: Package mapping Ubuntu macOS Xcode Command Line Tools Python's built-in If you don't have the Apple compiler tools installed, run: Then: For your TLS-Fingerprinting C++ project on the M2 Mac, that's the setup I'd use.

Code Block 1 (text)

```
brew update

brew install cmake libpcap openssl redis python

brew services start redis

redis-cli ping
```

Code Block 2 (text)

PONG

Code Block 3 (text)

```
xcode-select --install
```

Code Block 4 (text)

```
brew update  
brew install cmake libpcap openssl redis python  
brew services start redis  
redis-cli ping
```